

Q.1 Check the current SELinux status on your Linux system using the `getenforce` and `sestatus` commands, and write down what each output means.

ANS : Run these two commands on your CentOS system:

`getenforce`

and:

`sestatus`

What the outputs mean

1. `getenforce`

It will show one of these:

- Enforcing → SELinux is **enabled and actively blocking/allowing actions according to its security policies**.
- Permissive → SELinux is **enabled but only logs policy violations; it does not block them**.
- Disabled → SELinux is **completely turned off**.

2. `sestatus`

This gives more detailed information, for example:

```
SELinux status:                enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux mount point:            /sys/fs/selinux
SELinux policy root:            /etc/selinux
Current mode:                   enforcing
Mode from config file:          enforcing
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      ...
```

For your practical write-up

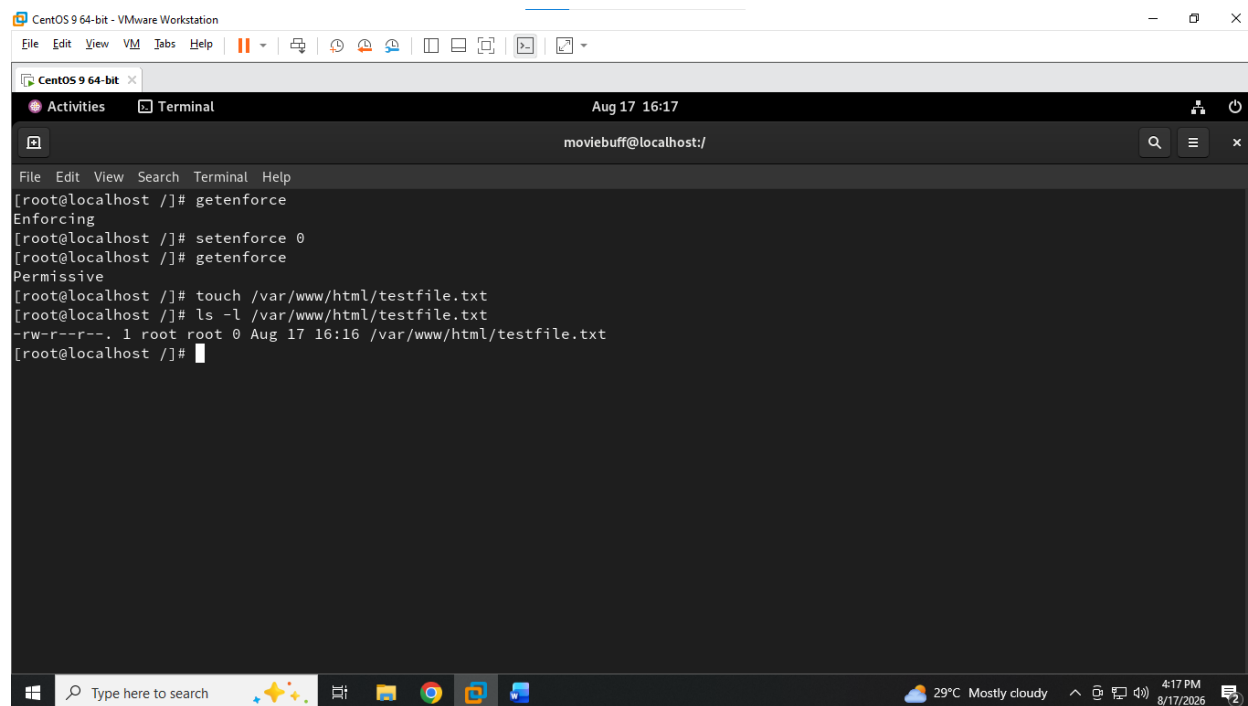
Module-5 : Managing SELINUX

Guided by : Vipul mistri

I checked the SELinux status using the `getenforce` and `sestatus` commands. The `getenforce` command displays the current SELinux operating mode, while `sestatus` provides detailed information about SELinux, including whether it is enabled, its current mode, and the loaded security policy. If the output shows **Enforcing**, SELinux is actively enforcing its security policies; **Permissive** means violations are logged but not blocked; and **Disabled** means SELinux is not running.

Q.2 Switch SELinux to permissive mode temporarily using the `setenforce` command, then try creating a new file in `/var/www/html` and note any differences in error messages or permissions compared to enforcing mode, Hint: Use `setenforce 0` for permissive and `setenforce 1` to return to enforcing.

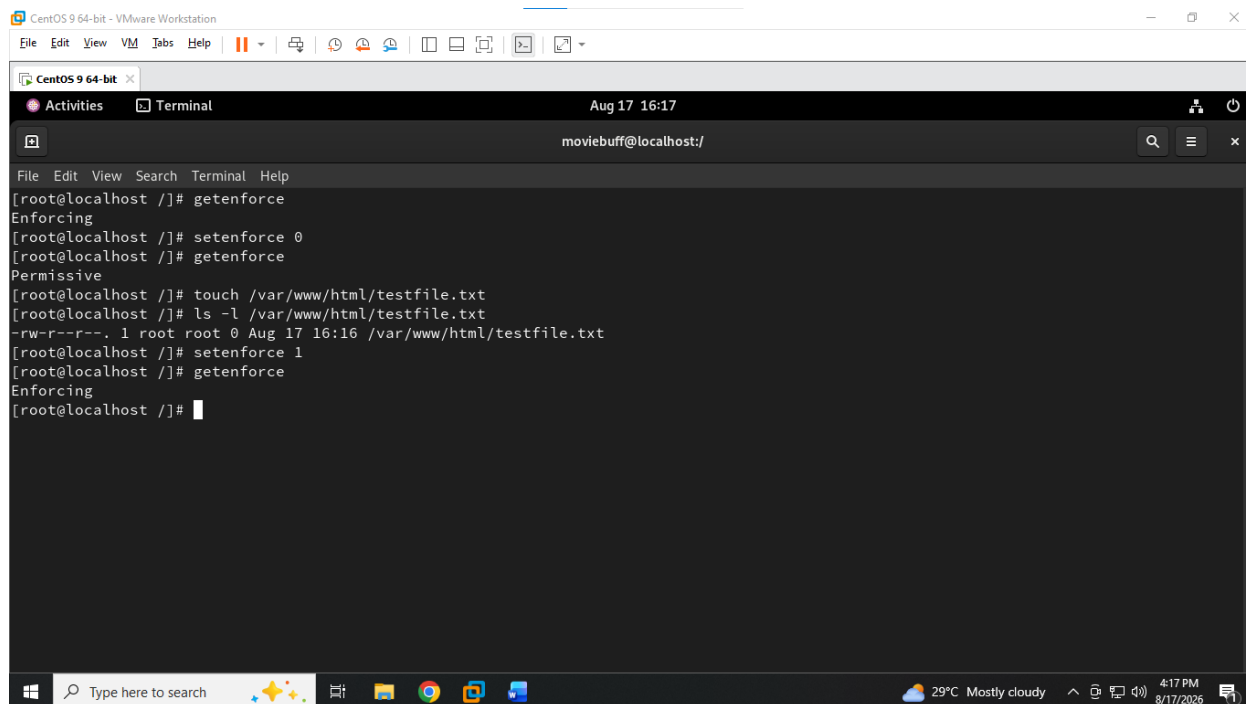
ANS :



```
CentOS 9 64-bit - VMware Workstation
File Edit View VM Tabs Help
CentOS 9 64-bit x
Activities Terminal Aug 17 16:17
moviebuff@localhost:/
File Edit View Search Terminal Help
[root@localhost ~]# getenforce
Enforcing
[root@localhost ~]# setenforce 0
[root@localhost ~]# getenforce
Permissive
[root@localhost ~]# touch /var/www/html/testfile.txt
[root@localhost ~]# ls -l /var/www/html/testfile.txt
-rw-r--r--. 1 root root 0 Aug 17 16:16 /var/www/html/testfile.txt
[root@localhost ~]#
```

Module-5 : Managing SELINUX

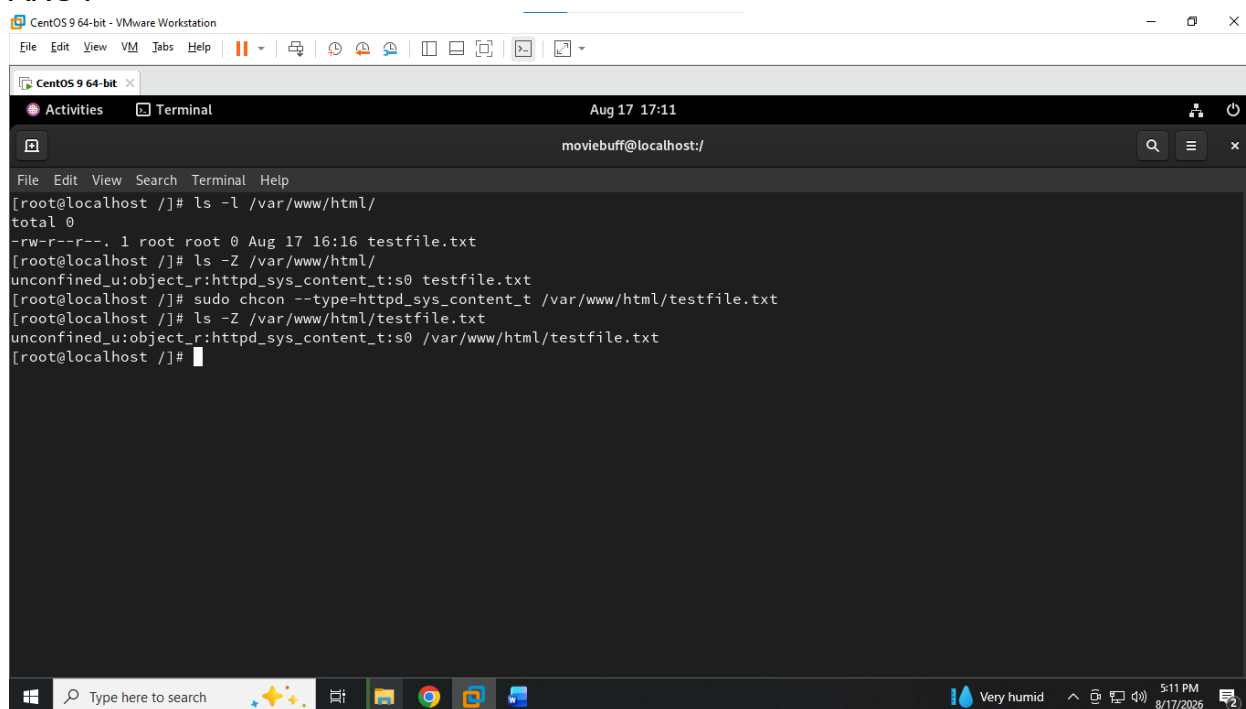
Guided by : Vipul mistri



```
CentOS 9 64-bit - VMware Workstation
File Edit View VM Tabs Help
CentOS 9 64-bit x
Activities Terminal Aug 17 16:17
moviebuff@localhost:/
File Edit View Search Terminal Help
[root@localhost ~]# getenforce
Enforcing
[root@localhost ~]# setenforce 0
[root@localhost ~]# getenforce
Permissive
[root@localhost ~]# touch /var/www/html/testfile.txt
[root@localhost ~]# ls -l /var/www/html/testfile.txt
-rw-r--r--. 1 root root 0 Aug 17 16:16 /var/www/html/testfile.txt
[root@localhost ~]# setenforce 1
[root@localhost ~]# getenforce
Enforcing
[root@localhost ~]#
```

Q.3 List all files in /var/www/html and use the ls -Z command to display their SELinux security contexts. Change the context of one file to httpd_sys_content_t using chcon and verify the change, Hint: Use chcon --type=httpd_sys_content_t filename

ANS :



```
CentOS 9 64-bit - VMware Workstation
File Edit View VM Tabs Help
CentOS 9 64-bit x
Activities Terminal Aug 17 17:11
moviebuff@localhost:/
File Edit View Search Terminal Help
[root@localhost ~]# ls -l /var/www/html/
total 0
-rw-r--r--. 1 root root 0 Aug 17 16:16 testfile.txt
[root@localhost ~]# ls -Z /var/www/html/
unconfined_u:object_r:httpd_sys_content_t:s0 testfile.txt
[root@localhost ~]# sudo chcon --type=httpd_sys_content_t /var/www/html/testfile.txt
[root@localhost ~]# ls -Z /var/www/html/testfile.txt
unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/html/testfile.txt
[root@localhost ~]#
```

Q.4 A user is unable to upload images to a folder for their food review blog (like Zomato) hosted in /srv/foodie/images. Use the audit2why and audit2allow tools to analyze SELinux denial logs and suggest a solution to allow uploads. Hint: Check /var/log/audit/audit.log for recent denials.

ANS : For this SELinux practical, you can analyze the denial and create a policy for the upload directory as follows.

1. Check recent SELinux denials

```
sudo ausearch -m AVC,USER_AVC -ts recent
```

You can also inspect the audit log directly:

```
sudo tail -n 50 /var/log/audit/audit.log
```

2. Analyze the denial with audit2why

Pipe the AVC denials into:

```
sudo ausearch -m AVC,USER_AVC -ts recent | audit2why
```

This explains **why SELinux denied the upload operation** and may suggest an appropriate SELinux boolean or policy change.

3. Check the directory context

```
ls -Zd /srv/foodie/images
```

For an Apache upload directory, a writable SELinux type is typically appropriate:

```
httpd_sys_rw_content_t
```

Set it persistently with:

```
sudo semanage fcontext -a -t httpd_sys_rw_content_t "/srv/foodie/images(/.*)?"
```

```
sudo restorecon -Rv /srv/foodie/images
```

Verify:

```
ls -Zd /srv/foodie/images
```

4. Use audit2allow if a custom policy is actually required

First collect the relevant denial:

```
sudo ausearch -m AVC,USER_AVC -ts recent > /tmp/foodie-avc.log
```

Review what it proposes:

```
sudo audit2allow -w -i /tmp/foodie-avc.log
```

If the denial is legitimate and there is no suitable existing SELinux type/boolean, generate a local policy:

```
sudo audit2allow -M foodie_upload < /tmp/foodie-avc.log
```

Review the generated policy before installing it. If it contains only the expected upload-related permissions, install it with:

```
sudo semodule -i foodie_upload.pp
```

5. Test the upload again

Try uploading an image through the food-review website and then check:

```
ls -l /srv/foodie/images
```

If the upload succeeds, check for new AVC denials:

```
sudo ausearch -m AVC -ts recent
```

Recommended solution

Prefer the SELinux labeling solution over blindly using audit2allow. For an Apache application that genuinely needs to write uploaded images, label /srv/foodie/images as httpd_sys_rw_content_t and restore the context:

```
sudo semanage fcontext -a -t httpd_sys_rw_content_t "/srv/foodie/images(/.*)?"
```

```
sudo restorecon -Rv /srv/foodie/images
```

audit2allow should be used only after examining the denial and confirming that the requested access is legitimate; blindly allowing every generated rule can weaken SELinux security.